DM587

Scientific Programming

Lecture 04: Visualization and pandas

Sai Ganesh Nagarajan

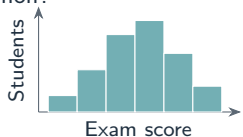
28 October 2026

Data analysis and plotting

Data analysis uses observations to answer questions. Plots make patterns, relationships and variation easier to see.

Histogram

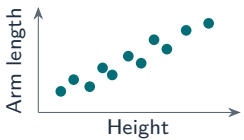
Which exam scores are common?



Counts values in intervals.

Scatter plot

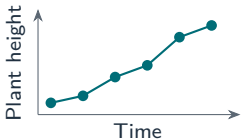
Do taller people have longer arms?



Compares two measurements per person.

Line / function plot

How does a plant grow over time?



Follows change; also plots functions such as $y = x^2$.

Choose the plot to match the question.

Illustrative examples, not measured data.

Tools

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

- NumPy: numerical arrays and calculations.
- Matplotlib: figures, plotting panels and visual marks.
- Pandas: labelled tables and time series.

The try-it bundle includes the CSV. The notebook runs without downloading weather data.

Install packages in your notebook's environment. Statsmodels is needed only for the forecasting section.

01

Visualization

From mathematical functions to weather data.

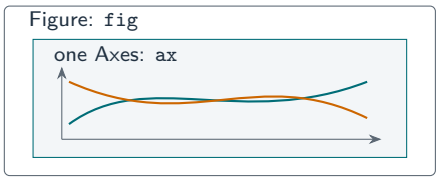
Section reading

Matplotlib quick start: Figure/Axes, plotting, labels, legends and subplots

Figure and Axes

A **Figure** is the whole canvas. An **Axes** object is one plotting panel.

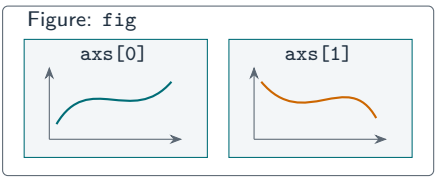
Two lines in one panel




```
fig, ax = plt.subplots()
ax.plot(x, y1)
ax.plot(x, y2)
```

Both curves share the same x- and y-axes.

Two separate subplots




```
fig, axs = plt.subplots(1, 2)
axs[0].plot(x, y1)
axs[1].plot(x, y2)
```

Each panel has its own x- and y-axes.

General rule: *m* rows × *n* columns give *mn* Axes objects in one Figure.

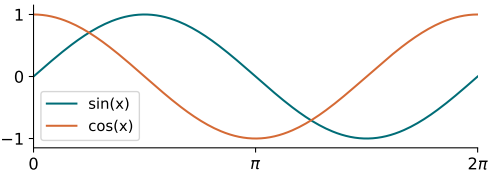
```
fig, axs = plt.subplots(m, n, squeeze=False)
axs[i, j].plot(x, y)     # Row i, column j; indices start at 0
```

`squeeze=False` keeps `axs` two-dimensional, even for one row or column.

Functions

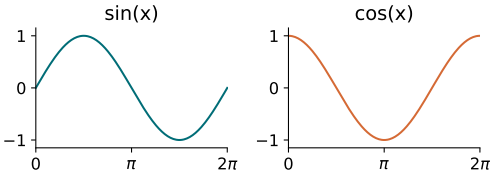
```
x = np.linspace(0, 2 * np.pi, 200)
```

One panel, two curves




```
fig, ax = plt.subplots()
ax.plot(x, np.sin(x), label="sin(x)")
ax.plot(x, np.cos(x), label="cos(x)")
ax.legend()
plt.show()
```

Two panels, one curve each




```
fig, axs = plt.subplots(1, 2)
axs[0].plot(x, np.sin(x))
axs[1].plot(x, np.cos(x))
axs[0].set_title("sin(x)")
axs[1].set_title("cos(x)")
plt.show()
```

Use one panel to compare curves directly; use subplots to view them separately.

Questions

We will practise data analysis using publicly available weather data near Odense, covering 2006–2025.

What can we learn from this record?

- How does temperature change over the course of a year?
- Which temperatures are common, and which are unusual?
- Do days with higher minimum temperatures also have higher maximum temperatures?
- Can past temperature data help predict future temperatures?

Question $\rightarrow$ data $\rightarrow$ plot $\rightarrow$ interpretation.

02

Pandas

Understand labelled tables before selecting and plotting their data.

Section reading

Pandas tutorials: read tables, select subsets, derived columns, summaries and time series

DataFrame

A DataFrame is a table with labelled rows and columns.

Index	date	temp_mean	temp_min	temp_max	rain_mm
0	2006-01-01	1.3	-0.2	2.2	8.0
1	2006-01-02	-1.4	-5.5	1.7	0.0
2	2006-01-03	-2.9	-5.0	-0.8	0.0

- A column has a name, such as `temp_mean`.
- A row represents one day; its index label is initially 0, 1, 2, ...
- Selecting one column gives a Series with the same row labels.

Columns may have different types: dates, numbers or text.

The data

Daily weather near Odense, 2006–2025: 7,305 days

```
data = pd.read_csv("odense-weather.csv", parse_dates=["date"])
print(data.columns.tolist())
```

```
['date', 'temp_mean', 'temp_min', 'temp_max', 'rain_mm']
```

```
print(data.head(3).to_string(index=False))  # head(3): first 3 rows
```

date	temp_mean	temp_min	temp_max	rain_mm
2006-01-01	1.3	-0.2	2.2	8.0
2006-01-02	-1.4	-5.5	1.7	0.0
2006-01-03	-2.9	-5.0	-0.8	0.0

One row per day. Temperatures are in °C; rainfall is the daily total in mm.

head = beginning: `data.head(3)` selects the first three rows; `data.head()` selects five.

iloc = integer positions: `data.iloc[0:3, :]` selects the same rows and all columns.

Positions start at 0; the stop 3 is excluded. Use `iloc[rows, columns]` for general positional slicing.

Open-Meteo / Copernicus ERA5; CC BY 4.0. Column names shortened; values unchanged.

Date index

Use the table already loaded; make its dates the row labels:

```
weather = data.set_index("date").sort_index()
print(weather.head(3).to_string())
```

Date index	temp_mean	temp_min	temp_max	rain_mm
2006-01-01	1.3	-0.2	2.2	8.0
2006-01-02	-1.4	-5.5	1.7	0.0
2006-01-03	-2.9	-5.0	-0.8	0.0

- `parse_dates` in `read_csv` converted date text to dates.
- `set_index` makes those dates the row labels; `sort_index` orders them.
- `weather.shape` is `(7305, 4)`; `date` is now the index, not a data column.

Series and DataFrame

```
weather["temp_mean"]      # Series: one column
weather[["temp_mean"]]    # DataFrame: one-column table
weather.iloc[0]           # Series: one row
weather.iloc[[0]]         # DataFrame: one-row table
```

Selection	Result	Shape
One column name	Series	`(7305,)`
A list with one column name	DataFrame	`(7305, 1)`
One row position	Series	`(4,)`
A list with one row position	DataFrame	`(1, 4)`

A row Series uses the column names as its index and contains that row's values.

A Series is one-dimensional; a DataFrame is a two-dimensional table. The extra brackets preserve the table structure.

Labels and positions

```
weather.loc["2024-01-01":"2024-01-03", ["temp_mean"]]
weather.iloc[:3]
```

`loc`: labels

Selects 1, 2 and 3 January 2024.

The end date is **included**.

`iloc`: positions

Selects rows at positions 0, 1 and 2.

The stop position 3 is **excluded**.

Here those first three positions are in January 2006, not January 2024.

Filtering

Which July 2024 days had at least 5 mm of rain?

```
july = weather.loc["2024-07-01":"2024-07-31"]
wet = july.loc[july["rain_mm"] >= 5, ["temp_mean", "rain_mm"]]
```

For two conditions:

```
mask = (july["temp_mean"] > 18) & (july["rain_mm"] >= 5)
july.loc[mask]
```

Use `&` with parentheses for elementwise conditions, rather than `and`.

Derived columns

```
weather["temp_range"] = weather["temp_max"] - weather["temp_min"]
```

temp_max	temp_min	temp_range
2.2, 1.7, -0.8	-0.2, -5.5, -5.0	2.4, 7.2, 4.2

- This adds `temp_range` as a new column in `weather`; if it exists, its values are replaced.
- Subtract columns directly; no loop over rows is needed.
- Subtraction pairs values with the same date, even if selected rows are in a different order.
- The change is in memory; the original CSV file is unchanged.

Missing data

Check	Question
`weather.isna().sum()`	Are values missing?
`weather.index.duplicated().sum()`	Does a date occur twice?
Compare with a full date range	Are entire days absent?

This extract has no missing values, duplicate dates or absent days.

In general, `NaN` means unknown. Replacing missing values with chosen numbers changes the data itself and can affect the conclusions.

`mean()` normally skips missing values; `count()` tells you how many values contributed.

Use a deliberate rule for missing data before comparing periods.

Summary statistics

```
weather[["temp_mean", "rain_mm"]].describe().round(2)
```

`.round(2)` rounds the summary values to two decimal places, e.g. $9.587\ldots \to 9.59$. It leaves `weather` unchanged.

	Mean temperature (°C)	Rainfall (mm)
count	7305	7305
mean	9.59	1.84
std	6.47	3.49
min	-10.10	0.00
25%	4.80	0.00
50%	9.40	0.20
75%	15.00	2.20
max	25.90	58.40

50% is the median; 25% and 75% mark the middle half of the values.

These summaries pool all days. They hide the order of days and the seasonal cycle.

Plot inputs

Use the table `data` that we have just loaded:

```
dates = data["date"]
mean_temp = data["temp_mean"].to_numpy()
```

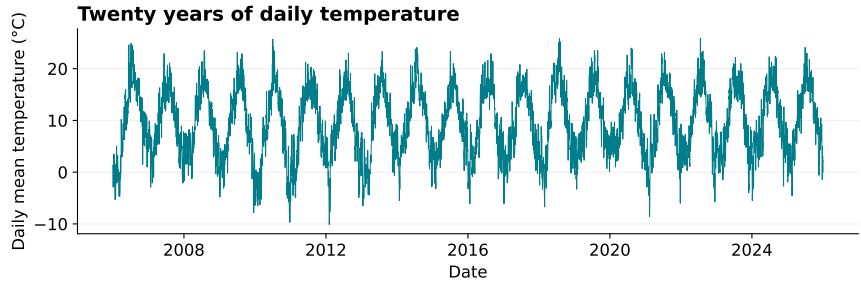
- `dates` selects the date column, already converted by `parse_dates`. The plot can therefore use a calendar axis.
- `to_numpy()`: extracts the numerical column as a NumPy array.

x-values: dates	y-values: daily mean temperature
2006-01-01	1.3
2006-01-02	-1.4
2006-01-03	-2.9

`ax.plot(dates, mean_temp)` plots one temperature against each date.

Entries at the same position come from the same row: one date–temperature pair per day.

Line plot

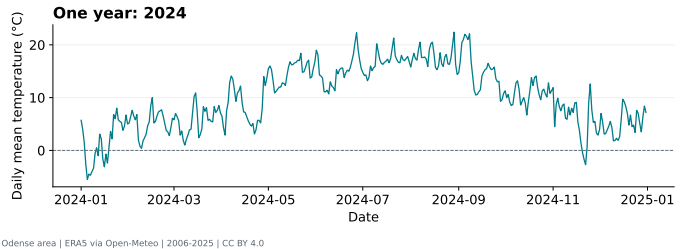


Odense area | ERA5 via Open-Meteo | 2006-2025 | CC BY 4.0

```
fig, ax = plt.subplots()
ax.plot(dates, mean_temp)
plt.show()
```

What repeats? What is difficult to see at this scale?

Scale




```
in_2024 = dates.dt.year == 2024
fig, ax = plt.subplots()
ax.plot(dates[in_2024], mean_temp[in_2024])
plt.show()
```

`dates.dt.year` extracts each date's year. The comparison creates a Boolean mask: `True` for 2024, `False` otherwise.

Use the same mask for dates and temperatures to keep their pairs together.

Monthly averages

Prepare one row per month before comparing the three temperature columns.

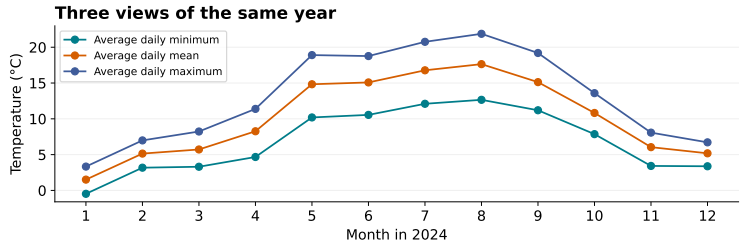
```
months = dates.dt.month.to_numpy()
month_numbers = np.arange(1, 13)
columns = ["temp_min", "temp_mean", "temp_max"]
monthly_2024 = np.array([
    data.loc[in_2024 & (months == m), columns].mean()
    for m in month_numbers])
```

- `months` labels each day from 1 to 12; `in_2024` selects the year.
- For each month, select its days and average each temperature column.

Month	Mean of daily minima	Mean temperature	Mean of daily maxima
January	-0.47	1.51	3.33
February	3.18	5.14	6.99
March	3.31	5.72	8.23

The resulting array has 12 month-rows and 3 temperature-columns.

Several lines

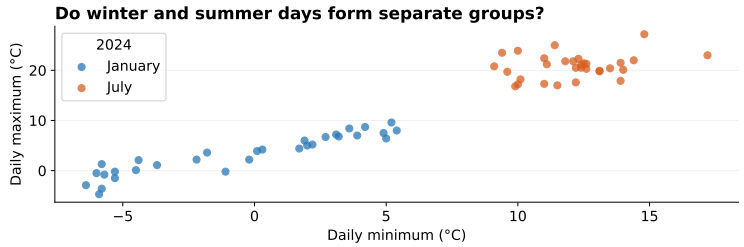



```
fig, ax = plt.subplots()
for j, name in enumerate(columns):
    ax.plot(month_numbers, monthly_2024[:, j], label=name)
ax.legend()
```

`j` selects a column in the monthly array; `name` supplies its legend label. The previous slide prepared both.

Each minimum-temperature point averages daily minima; it is not the month's lowest temperature.

Scatter

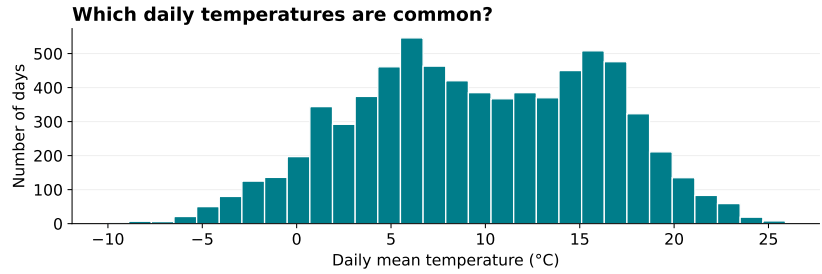



```
for month, name in [(1, "January"), (7, "July")]:
    selected = in_2024 & (months == month)
    ax.scatter(min_temp[selected], max_temp[selected], label=name)
ax.legend(title="2024")
```

Start with `fig, ax = plt.subplots()`. Each call plots one month in a different colour.

One point per day. Do January and July form two groups? Compare both the minimum and maximum temperatures.

Histogram

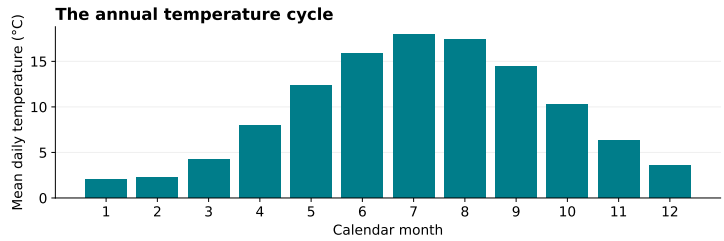



```
fig, ax = plt.subplots()
ax.hist(mean_temp, bins=30, edgecolor="white")
```

`mean_temp` is the full daily-mean column extracted earlier. `bins=30` divides its range into 30 intervals.

Bar heights count days in each interval; their dates are not shown.

Bar chart

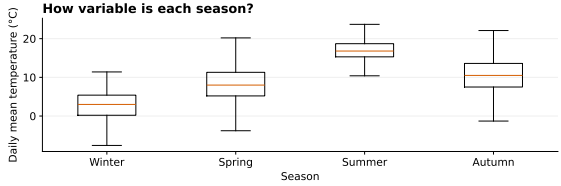



```
month_profile = np.array([
    mean_temp[months == m].mean() for m in month_numbers])
fig, ax = plt.subplots()
ax.bar(month_numbers, month_profile)
```

`months` labels each day from 1 to 12; `month_numbers` is 1 through 12. `month_profile` contains their twelve averages, pooling all years.

One category per month; heights are average daily temperatures, not counts.

Box plot

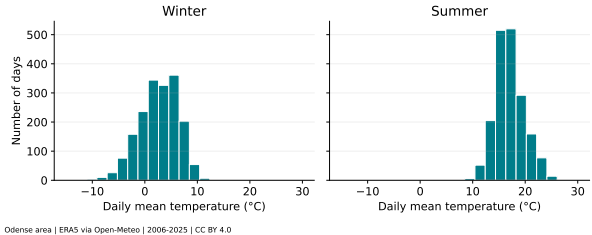



```
season_names = ["Winter", "Spring", "Summer", "Autumn"]
season_months = [(12, 1, 2), (3, 4, 5), (6, 7, 8), (9, 10, 11)]
season_values = [mean_temp[np.isin(months, group)]
                 for group in season_months]
fig, ax = plt.subplots()
ax.boxplot(season_values, showfliers=False)
ax.set_xticklabels(season_names)
```

`np.isin` selects each season's days; `season_values` stores the four resulting arrays.

Line: median. Box: middle 50%. Whiskers reach data within 1.5 box lengths; outlying points are hidden.

Subplots

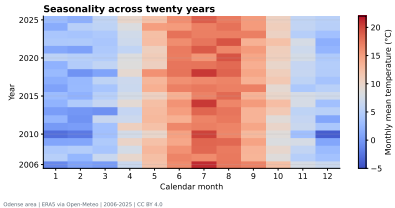



```
winter, summer = season_values[0], season_values[2]
bins = np.linspace(-15, 30, 24)
fig, axs = plt.subplots(1, 2, sharex=True, sharey=True)
for ax, values, label in zip(axs, [winter, summer], ["Winter", "Summer"]):
    ax.hist(values, bins=bins, edgecolor="white")
    ax.set_title(label)
```

Reuse the winter and summer arrays from the previous slide. `bins` supplies the same interval edges; `zip` pairs each panel with its data and title.

Shared axis limits and bins make the distributions easier to compare.

Heatmap

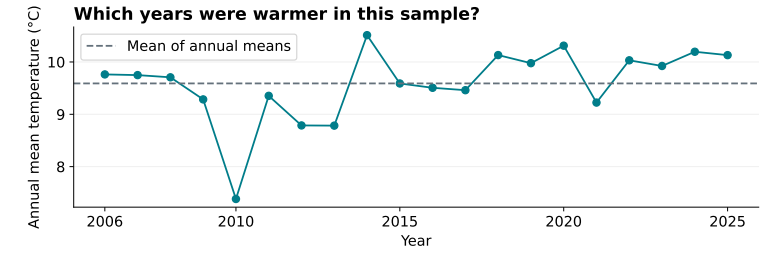



```
years = dates.dt.year.to_numpy()
unique_years = np.arange(2006, 2026)
monthly_grid = np.array([
    [mean_temp[(years == y) & (months == m)].mean()
     for m in month_numbers] for y in unique_years])
fig, ax = plt.subplots()
image = ax.imshow(monthly_grid, aspect="auto", origin="lower",
                  extent=[0.5, 12.5, 2005.5, 2025.5], cmap="coolwarm")
fig.colorbar(image, ax=ax, label="Monthly mean temperature (C)")
```

`monthly_grid` has 20 year-rows and 12 month-columns. Each entry averages that year/month's days.

`extent` labels the horizontal and vertical coordinates.

Annual means

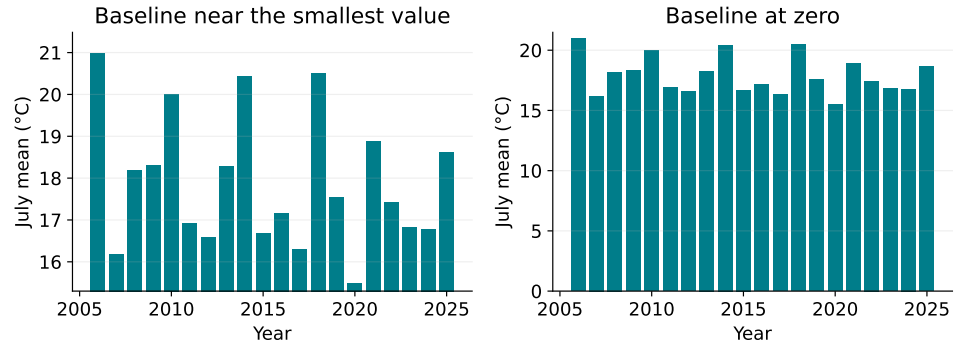



```
annual_values = np.array([
    mean_temp[years == y].mean() for y in unique_years])
fig, ax = plt.subplots()
ax.plot(unique_years, annual_values, marker="o")
```

`years` is each day's year; `unique_years` lists 2006–2025. `annual_values` holds one average per year.

Averaging across a year reduces seasonality. This local record alone does not establish a climate trend.

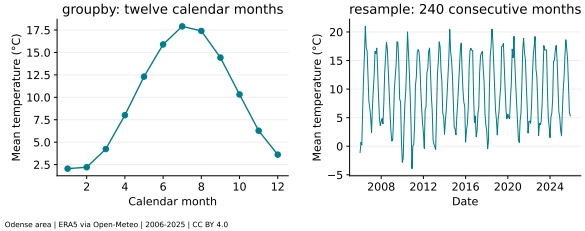
Choosing a baseline



Odense area | ERA5 via Open-Meteo | 2006-2025 | CC BY 4.0

Both panels use exactly the same data. For bars, a zero baseline preserves the meaning of length.

Grouping and resampling



Groupby: collect rows with the same label, then summarise each group—for example, a mean per city, year or month.

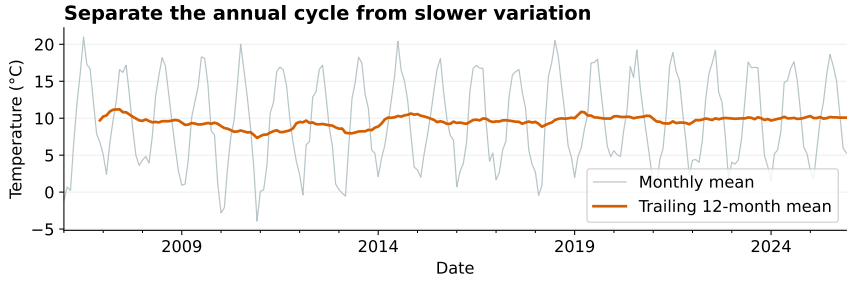
```
profile = weather["temp_mean"].groupby(weather.index.month).mean()
monthly = weather["temp_mean"].resample("MS").mean()
```

Left: group by month number
January 2006, January 2007, etc. go into one group. Average their daily values.
12 results: one for each month number.

Right: resample into calendar months
Average each month's days separately: January 2006, February 2006, etc.
240 results: one per month across 20 years.

Resampling needs a date index; MS labels each full-month average by its first date.

Rolling average



Odense area | ERA5 via Open-Meteo | 2006-2025 | CC BY 4.0

```
smooth = monthly.rolling(12, min_periods=12).mean()
monthly.plot(ax=ax)
smooth.plot(ax=ax)
```

Current month plus the previous eleven; the first eleven results are missing. Smoothing is not forecasting.

Seasonality and trend

- **Seasonality**: a pattern that repeats with the calendar.
- **Slower change**: variation that remains after averaging across seasons.
- **Anomaly**: a departure from a specified reference pattern.

03

Forecasting

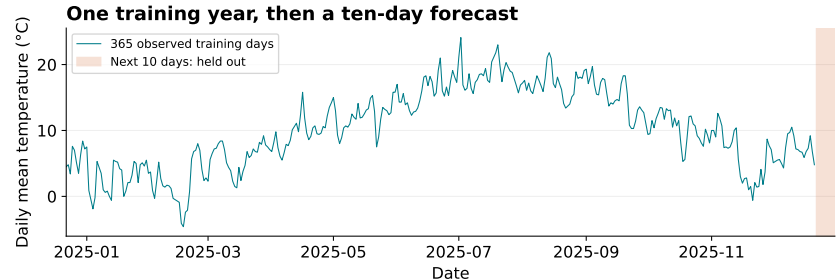
Predicting the future based on the past.

Section reading

Statsmodels: ARIMA / ARMA

Out-of-sample forecasts and prediction intervals

Ten days ahead

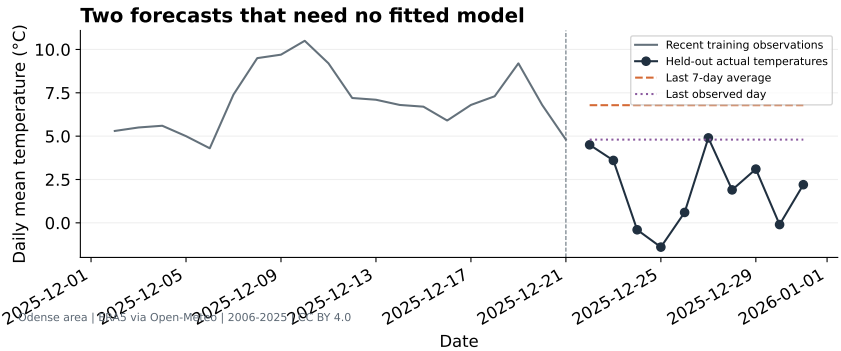


Odense area | ERA5 via Open-Meteo | 2006-2025 | CC BY 4.0

- Fit using 365 days: 22 December 2024–21 December 2025.
- Predict the next ten days: 22–31 December 2025.
- Keep those ten observations hidden until we check the predictions.

Each value is one day's mean temperature. The earlier monthly averages were for exploring the data.

A starting point



Two simple rules: predict the last observed temperature, or the average of the preceding seven days. Keep each prediction fixed across the ten days.

A prediction model

A model is a function that turns past observations into a prediction:

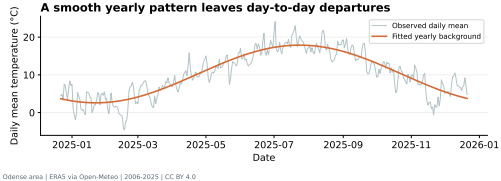
$$\underbrace{\widehat{T}_{t+h}}_{\text{predicted temperature}} = f_h\left(\underbrace{T_1, \ldots, T_t}_{\text{past data}}; \underbrace{\theta}_{\text{parameters / weights}}\right).$$

- T_t is today's observed temperature; h is how many days ahead we predict. The hat marks a prediction.
- θ is a collection of adjustable numbers: it controls how the function uses the past data.
- **Fit**: choose these numbers using past observations, aiming to make prediction errors small.
- **Forecast**: keep the fitted numbers fixed and apply the rule to future days, whose temperatures are unknown.

A simple choice

$\widehat{T}_{t+1} = w_1 T_t + w_2 T_{t-1}$ uses the last two days, with parameters $\theta = (w_1, w_2)$. Equal weights $w_1 = w_2 = \frac{1}{2}$ give their average.

The yearly curve



Choose a smooth shape that repeats once a year:

$$s_t = c_0 + c_1 \sin\left(\frac{2\pi t}{365.25}\right) + c_2 \cos\left(\frac{2\pi t}{365.25}\right).$$

- t counts days from the first training date. c_0 sets the level; c_1, c_2 control the size and timing of the seasonal rise and fall.
- Fit the three coefficients using the 365 training temperatures: choose them to minimise $\sum_{t=0}^{364}(T_t - s_t)^2$ (least squares). The fitting code is in the try-it notebook.
- Keep them fixed and evaluate the same curve at future dates. ARMA will model the remaining departures $r_t = T_t - s_t$.

ARMA

Goal: fit a rule to past temperatures and use it to predict future days.

$$T_t = \underbrace{s_t}_{\text{yearly pattern}} + \underbrace{r_t}_{\text{departure from that pattern}}.$$

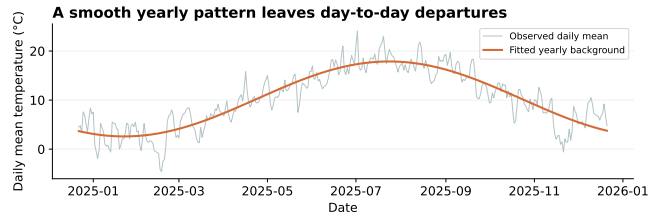
Fit a smooth yearly curve s_t using the training year. Then model its departures:

$$r_t = \underbrace{ar_{t-1}}_{\text{AR: yesterday's departure}} + \underbrace{be_{t-1}}_{\text{MA: yesterday's prediction error}} + \underbrace{e_t}_{\text{today's new surprise}}.$$

- t counts days; a and b are numbers learned from past data.
- e_t is the observed departure minus its one-day prediction.
- Assume this relationship remains useful over the next ten days.

ARMA(1,1) uses one previous departure and one previous error. Its MA term is not a rolling average of temperatures.

Fit and predict

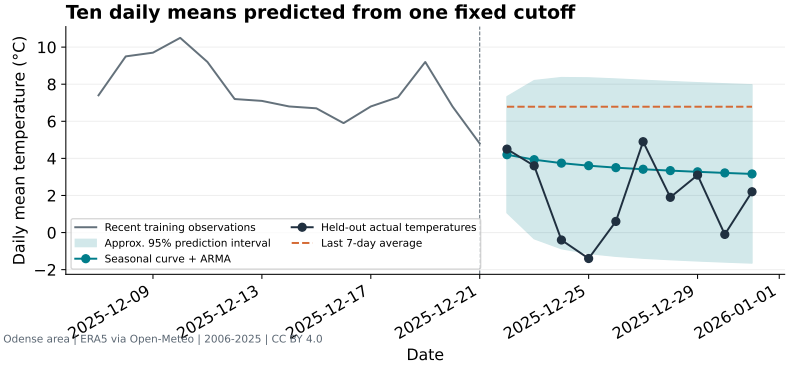


`train_residual` contains training temperatures minus the fitted yearly curve.

```
from statsmodels.tsa.arima.model import ARIMA
fit = ARIMA(train_residual, order=(1, 0, 1), trend="n").fit()
result = fit.get_forecast(steps=10)
```

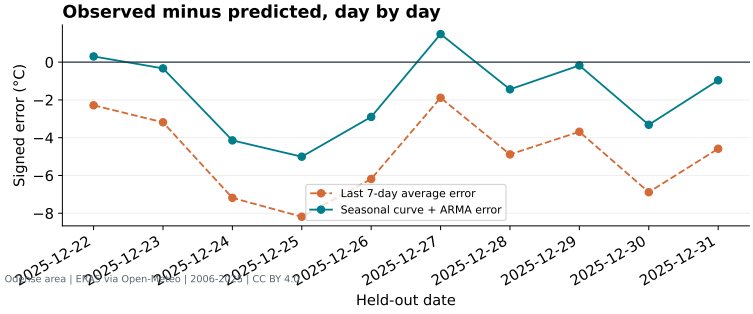
Add the yearly curve back to the predicted departures. The middle 0 means no differencing; ARIMA(1,0,1) implements ARMA(1,1). Full preparation is in the try-it notebook.

Predictions



One point is one day's mean temperature. The approximate 95% interval describes model uncertainty; it omits uncertainty in the fitted yearly curve. Actual errors need not increase with every extra day.

Errors



Open data area | Energy via Open-Meteo | 2006-2025 | CC BY 4.0

```
errors = test - prediction
mae = errors.abs().mean()
```

Positive: predicted too cold. Negative: predicted too warm. MAE is the average absolute miss across the ten hidden days.

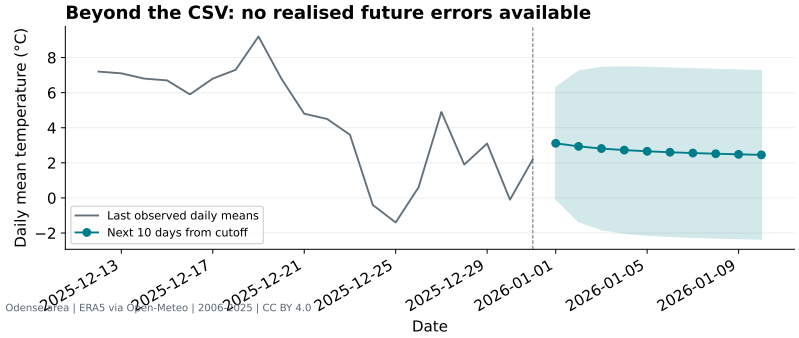
Comparison

Method	MAE (°C)	RMSE (°C)
Last seven days' average	4.90	5.31
Last observed day	2.93	3.56
Yearly curve alone	1.99	2.54
Yearly curve + ARMA	2.01	2.59

- The combined model beats the recent average on these ten days.
- The yearly curve alone does slightly better: extra complexity need not help.
- RMSE gives larger misses more weight than MAE.

What can you conclude from these results?

Beyond the dataset



Try forecasting for yourself: use ARMA, a recent average, or your own simple rule based on past data. Change how far ahead you predict and compare the results on days with known temperatures.

Readings

- Matplotlib quick start: figures, Axes, labels, legends and subplots.
- Pandas tutorials: tables, selections, derived columns, summary statistics and time series.
- Statsmodels ARIMA: optional background for the forecasting example.
- Open-Meteo historical weather: dataset, variables and ERA5 model.

Weather data by Open-Meteo, CC BY 4.0. Contains modified Copernicus Climate Change Service information (2026). Neither the European Commission nor ECMWF is responsible for use of this information.